```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

10. Tutorial - Best Practices & Debugging

Kuzey Aydınhan | 30. January 2026

Contents

1. Overview

2. Best Practices

3. Debugging

4. Regex

Information

- Saalübung on ~~02.~~[1] 03. Feb.
  - Hints to solving the exercises
  - Q&A
- First Abschlussafg. release on 16. Feb.

[1] Correction @ 30.01. - Saalübung will be held on 03.

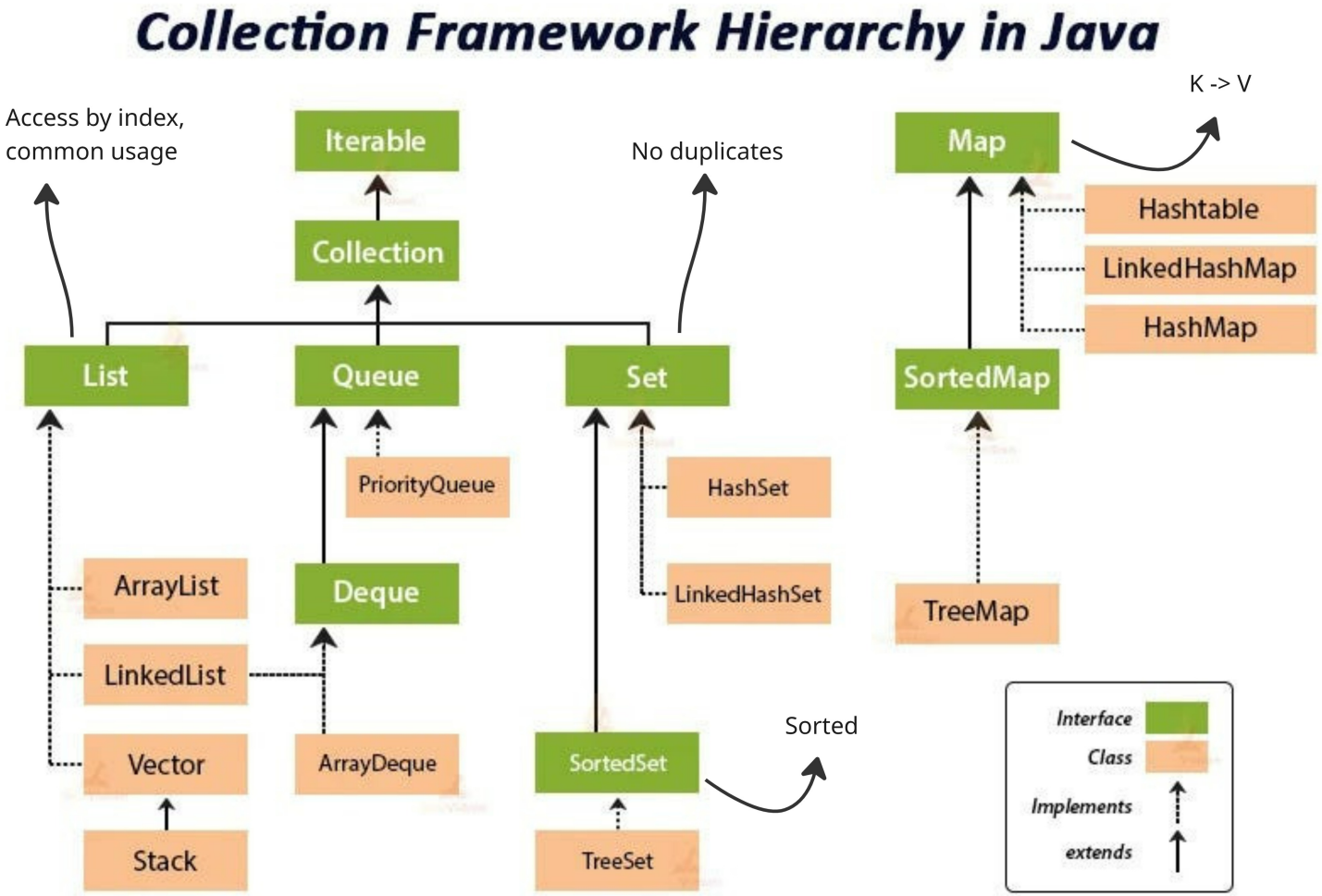
Exam Inspeciton

(Einsicht)

- It's not allowed to take photos!
- It's not allowed to inspect another exam
- If you have any questions just come over

Last Week

Java API



Last Week

Java API

`HashMap`
- No order
- Basic

`LinkedHashMap`
- Insertion order

`TreeMap`
- Sorted

`HashSet`
- No order
- Basic

`LinkedHashSet`
- Insertion order

`TreeSet`
- Sorted

List

- Lists only have `LinkedList` vs. `ArrayList`

Last Week

Optional<T>

- Null safe
- Easier to control

```
Optional<Item> getItem() {
    if (itemExists()) {
        return Optional.of(item);
    }
    return Optional.empty();
}

Optional<Item> foundItemOpt = getItem();

if (foundItemOpt.isPresent) {
    Item foundItem = foundItemOpt.get();
    foundItem.use()
}

```

Last Week

OOP Principles

1. **Encapsulation**
2. **Composition over Inheritance**
3. **Interfaces over Classes**
4. **SOLID Principles**

Last Week

SOLID

1. **S**ingle Responsibility: Each class only has 1 responsibility
2. **O**pen Closed: Components must be open to extensions but closed to changes
3. **L**iskov Substitution: Objects of superclass must be replaceable with subclasses
4. **I**nterface Segregation: Classes should not be forced to implement unnecessary methods by interfaces
5. **D**ependency Inversion: Abstracts should not depend on details

Last Week

Dependency Inversion

- Without dependency inversion:

```
class UserService {
    private MongoDB optionalDatabase = new MongoDB();
    private ElasticDB database = new ElasticDB();

    private UserRepository userRepo = new UserRepo(database);
    // private UserRepository userRepo = new UserRepo(optionalDatabase);

    // ...
}

```

- Hard to replace used database

Last Week

Dependency Inversion

- With dependency inversion it would look like this:

```
class UserService {
    private Repository repo;

    public UserService(Repository repository) {
        this.repository = repository;
    }

    // ...
}

```

Motivation

- Best Practices
- Debugging
- Regex

Best Practices

- Immutability
- Small and clean classes
- Optimize for the next person reading your code (e.g. future self)

- `instanceof`
- == and `equals()`
- Overwriting equals

Immutability

- The field/attribute can't be changed once created

- Whenever possible use **`final`** for fields
- Favor immutable objects

```java
class Student {
    private final String id;
    private final String name;
    private final List<Course> courses;

    private final Date birthday;

    // Instead of:
    private int age;
}

```

Small Classes

- Keep the class only specific to an object
- Single responsibility is important
- Remove unnecessary fields/methods

```java
class Student {
    private final String id;
    private final String name;
    private final Date birthday;
    // specific to a student no extra information

    // ...
}

```

Example

```
public class InputParser {
    public static Student parseStudent(input) {..}

    public static Move parseMove(input) {..}
}

public class ArgumentParser {
    public static Configuration parseConfig(args) {..}
}

public class UI {
    private bool isRunning;

    public void runLoop() {...}
}

public record Move (int moveX, int moveY) {}

```

Polymorphism

- Usually incorrect usage
- It is bad practice if you are using:
  → "If class A do a( ), if class B do b( )"

```java
if (person instanceof Intern) {
    person.pay(0);
} else if (person instanceof Manager) {
    person.pay(1000);
}

```

- Instead it should be replaced with polymorphie
- What happens when we add new roles?

Open-Closed

- Writing code this way also violates the open-closed principle
- See tutorial 9 slides

Polymorphism

Implementation

```java
abstract class Employee {
    public abstract void pay();
}

class Intern extends Employee {..}

class Manager extends Employee {..}

// With polymorphism
Employee person;
person.pay();

```

equals()

Feature	`==` (Equality Operator)	`.equals()` (Method)
Comparison Type	Compares **references** (memory locations)	Compares **content** (values inside objects)
Works on Primitives?	Yes	No
Works on Objects?	Yes, but only for reference comparison	Yes, for content comparison
Can Be Overridden?	No	Yes
Use Case	Check if two references point to the same object	Check if two objects have identical data

Overwriting `equals()`

Rules

1. `x.equals(x)` → `true`
2. `x.equals(y)` → `y.equals(x)`
3. `x.equals(y)`, `y.equals(z)` → `x.equals(z)`
4. `x.equals(y)` and update `x,y` equally → `x.equals(y)`
5. `x.equals(null)` → `false`

- If `x.equals(y)` then also `x.hashCode() == y.hashCode()`

Overwriting `equals()`

```java
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (other == null) return false;
    if (getClass() != other.getClass()) return false; // Or instanceof

    MyClass castObject = (MyClass) other;
    // Compare attributes
    boolean isEqualA = this.a == castObject.a;  // primitive
    boolean isEqualB = this.b == castObject.b;  // enum
    boolean isEqualC = this.c.equals(castObject.c);  // object
    boolean isEqualD = this.d.compareTo(castObject.d);  // comparable
    return isEqualA && isEqualB && ..;
}

```

Overwriting `equals()`

Inheritance

- With inheritance its messy to write `equals`
- You have to choose between:
  - `x.equals(y) == y.equals(x)` → symmetry
  - `parent.equals(child) == true` → Liskov substitution

- Symmetry uses `getClass()` in which case doesnt work with inheritance
- Liskov substitution uses `instanceof` which is risky and easy to violate

Overwriting `equals()`

Dilemma

Symmetry

```
x.equals(y) == y.equals(x)

if (getClass() != o.getClass()) return false;

```

- `parent.equals(child)` (or vice versa) never true

Liskov Substitution Principle

```
parent.equals(child) == true

if (!(o instanceof Parent)) return false;

```

- Breaks when child classes add new fields

Overwriting `equals()`

Solution

- Make `equals()` **final** for parent class

```
class Parent {
    @Override
    public final boolean equals(Object obj) {..}
}

```

Composition

- If you have to overwrite the `equals()` from parent class, you shouldn't be using inheritance!

BAD

```java
class Person {
    @Override
    public final boolean equals(Object obj) {..}
}

class Employee extends Person {
    private String id;

    // Equals now also needs to check for id
}

```

GOOD

```java
class Person {..}

class Employee {
    private Person person;
    private String id;
}

```

Overwriting `hashCode()`

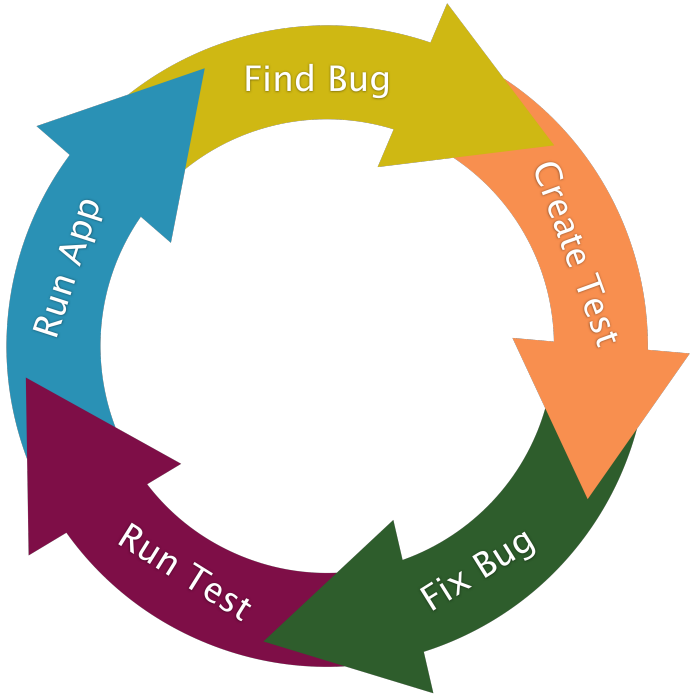
- Always overwrite `hashCode` and `equals` together!

```java
@Override
public int hashCode() {
    int result = 0;

    // For integer attributes
    result = 31 * result + i;

    // For double, float attributes
    result = 31 * result + (f == 0.0f ? 0 : Float.floatToIntBits(f));

    // For objects, enums
    result = 31 * result + (name == null ? 0 : name.hashCode());

    return result;
}

```

- In modern code:

```java
@Override
public int hashCode() {
    return Objects.hash(i, f, name);
}

```

Debugging



Finding Errors

- Static code analyse → meaningful but incomplete (only compiler errors)
- On end user → bad - shouldn't be the case
- Manually → takes too much effort and time
- Automated → best possible
  - Next week: how to write tests

Debugging

- As soon as an error found, the source must be identified and removed

Print Debugging

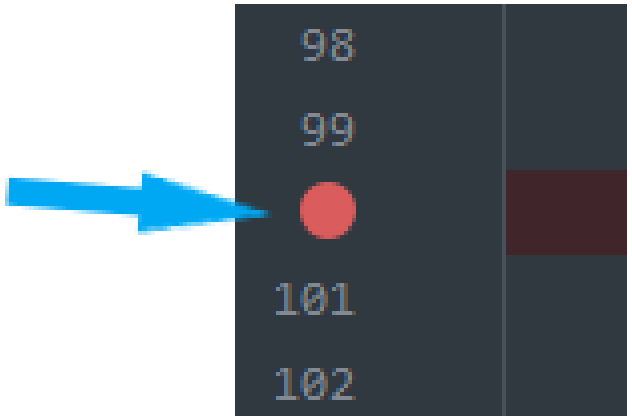
```java
List<Integer> myList = new ArrayList<>();
System.out.println(myList.toString());

myList.add(5);
System.out.println(myList.toString());

myList.addAll(otherList);
System.out.println(myList.toString());
System.out.println(myList.size());

myList.remove(5);
System.out.println(myList.toString());
System.out.println(myList.size());


```

- Too much effort
- Too much time
- Still might not work

Interactive Debugging

- Use IDE's interactive debugger
  - Set breakpoints
  - Inspect values

- Easier
- No code change



Interactive Debugging

- Eclipse Tutorial
- IntelliJ Tutorial

Interactive Debugging

Idea

- Main idea is to let the program execute step by step

- Set breakpoints at critical points
- Run program in debug mode
- Program will pause in the marked points
- Programmer has the chance to inspect the current state of program

Interactive Debugging

Tips

- Try out similar inputs and see what happens
- Try to find the line after where something is wrong
- Only check specific/small code parts and don't try to go through the whole code
- Ask another person
- Take breaks!
- Check for typos

Exercise 1

- In the tutorial folder there is a Musterlösung of HW 4
- The solution is failing
- Find the bug :)
- This interaction is the problem:

Current Interaction

```
> add-mailman Max;Mustermann;12345;password
OK
> authenticate 12345;password
OK
> get-mail Erika
Error, incorrect permissions.
```

Expected Interaction

```
> add-mailman Max;Mustermann;12345;password
OK
> authenticate 12345;password
OK
> get-mail Erika
Error, customer not found.
```

Regular Expressions

- Better definition in GBI and TGI
- Expressions that set definitions for matching texts and alphabets

- In java: powerful for checking `Strings`
- Can find RegEx testers and cheat sheets online: https://regexr.com/

Syntax Overview

Character classes

`.`	any character except newline
`\w \d \s`	word, digit, whitespace
`\W \D \S`	**not** word, digit, whitespace
`[abc]`	any of a, b, or c
`[^abc]`	not a, b, or c
`[a-g]`	character between a & g

Anchors

`^abc$`	start / end of the string
`\b \B`	word, not-word boundary

Escaped characters

`\. \* \\`	escaped special characters
`\t \n \r`	tab, linefeed, carriage return

Groups & Lookaround

`(abc)`	capture group
`\1`	backreference to group #1
`(?:abc)`	non-capturing group
`(?=abc)`	positive lookahead
`(?!abc)`	negative lookahead

Quantifiers & Alternation

`a* a+ a?`	0 or more, 1 or more, 0 or 1
`a{5} a{2,}`	exactly five, two or more
`a{1,3}`	between one & three
`a+? a{2,}?`	match as few as possible
`ab|cd`	match ab or cd

Example

- Useful for checking inputs

```
String input = scanner.next();
String regex = "^(add )([A-Z][a-z] ){2}$";
input.matches(regex);

```

```
String input = scanner.next();
String regex = "(-)?[\d]+$";
if (! input.matches(regex)) {
    System.out.println("Integer needed");
}
int i = Integer.parseInt(input);


```

Conclusion

- Prefer immutability
- Write small classes
- Prefer polymorphism
- Make the `equals` final if in parent class
- Always overwrite `hashCode` and `equals` together
- Use interactive debugging
- Regex

Exercise 1

- In `GetMailCommand` constructor, `AccountType.MAILMAN` is missing:

```
public GetMailCommand(PostalServiceSession session, PostalService service) {
    super("get-mail", session, service, List.of(AccountType.CUSTOMER), false);
}

```

- Corrected:

```
public GetMailCommand(PostalServiceSession session, PostalService service) {
    super("get-mail", session, service, List.of(AccountType.CUSTOMER, AccountType.MAILMAN), false);
}

```